PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course: Software Engineering | **Lab 7:** Fuzzing for Input Validation Bugs

Student Name:	Shridhar Umesh Golipalle	**SRN:**	PES1UG25AM814
Repository:	L6_L7_60_marathon_registration	**Folder:**	LAB7

1. Lab Overview & Objectives

The objective of Lab 7 is to employ **property-based fuzz testing** using Python's **Hypothesis** library to detect hidden input validation bugs in a marathon registration module (marathon_registration_parser.py). We then harden all three functions using defensive programming to ensure the application handles any input (including malformed, null, or extreme values) without crashing.

2. Screenshots: Buggy Module Execution & Fuzz Testing Failures

SS1: Buggy Module Execution (Valid Nominal Input)

```
PowerShell - SS1: Buggy Module Run (Valid Input)

PS C:\Users\91781\Desktop\PES\5th Sem\SE\Lab_7\L6_L7_60_marathon_registration\LAB7> python marathon_registration_parser.py
==== Running format_bib_name ====
Enter the name to print on the bib: John Doe
Bib name: JOHN DOE

==== Running parse_finish_time ====
Enter a finish time as H:MM:SS (e.g. 3:45:10): 3:45:10
Finish time in seconds: 13510

==== Running average_split ====
Enter your 5 km split times in minutes separated by spaces (e.g. 25 26 24): 25 26 24
Average split (minutes): 25.0

PS C:\Users\91781\Desktop\PES\5th Sem\SE\Lab_7\L6_L7_60_marathon_registration\LAB7>
```

Figure 1: SS1 — Buggy module executes cleanly on well-formed inputs but fails to defend against anomalies.

SS2: Completed Hypothesis Tests Failing on Buggy Module

```
PowerShell - SS2: Hypothesis Tests Failing on Buggy Module

PS C:\Users\91781\Desktop\PES\5th Sem\SE\Lab_7\L6_L7_60_marathon_registration\LAB7> pytest test_marathon_registration_parser.py
============================= test session starts =============================
platform win32 -- Python 3.10.8, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\91781\Desktop\PES\5th Sem\SE\Lab_7\L6_L7_60_marathon_registration\LAB7
plugins: anyio-4.13.0, hypothesis-6.168.5
collected 3 items

test_marathon_registration_parser.py FFF                                 [100%]

================================== FAILURES ===================================
_____________________ test_format_bib_name_no_crash ______________________
    return name.strip().upper()[:12]
E   AttributeError: 'NoneType' object has no attribute 'strip'
E   Failing test case: test_format_bib_name_no_crash(s=None)
_____________________ test_parse_finish_time_no_crash _____________________
    hours, minutes, seconds = text.split(':')
E   ValueError: not enough values to unpack (expected 3, got 1)
E   Failing test case: test_parse_finish_time_no_crash(s='')
E   Failing test case: test_parse_finish_time_no_crash(s=None)
_____________________ test_average_split_no_crash _____________________
    return sum(splits) / len(splits)
E   ZeroDivisionError: division by zero
E   Failing test case: test_average_split_no_crash(s='')
E   Failing test case: test_average_split_no_crash(s=None)
E   Failing test case: test_average_split_no_crash(s=':')
=========================== short test summary info ===========================
FAILED test_marathon_registration_parser.py::test_format_bib_name_no_crash - AttributeError
FAILED test_marathon_registration_parser.py::test_parse_finish_time_no_crash - ValueError / None
FAILED test_marathon_registration_parser.py::test_average_split_no_crash - ZeroDivisionError
============================== 3 failed in 13.85s ==============================
PS C:\Users\91781\Desktop\PES\5th Sem\SE\Lab_7\L6_L7_60_marathon_registration\LAB7>
```

Figure 2: SS2 — Hypothesis uncovers unhandled exceptions: AttributeError on None, ValueError on invalid format, and ZeroDivisionError on empty splits.

3. Screenshots: Hardened Module Execution & Passing Fuzz Tests

SS3: Execution of Fixed Module (Valid Nominal Input)

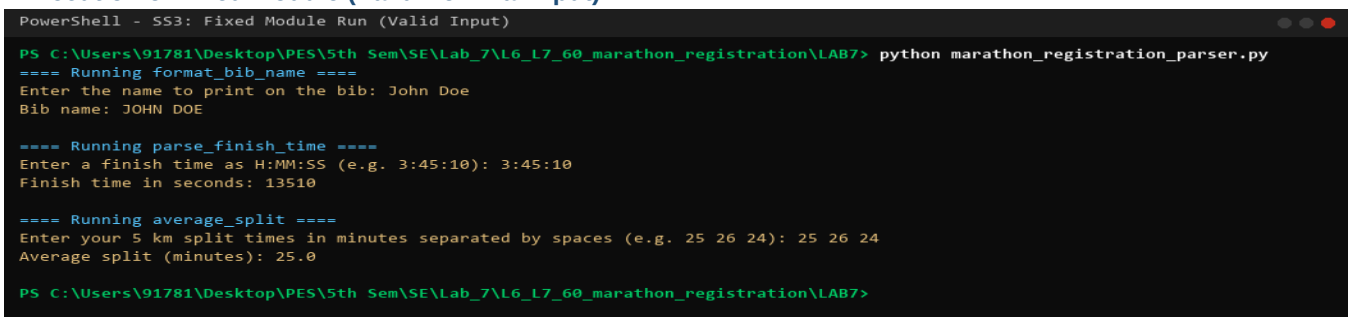


Figure 3: SS3 — Fixed module runs properly and returns identical expected outputs for valid runner details.

SS4: All Hypothesis Property Tests Passing on Fixed Module

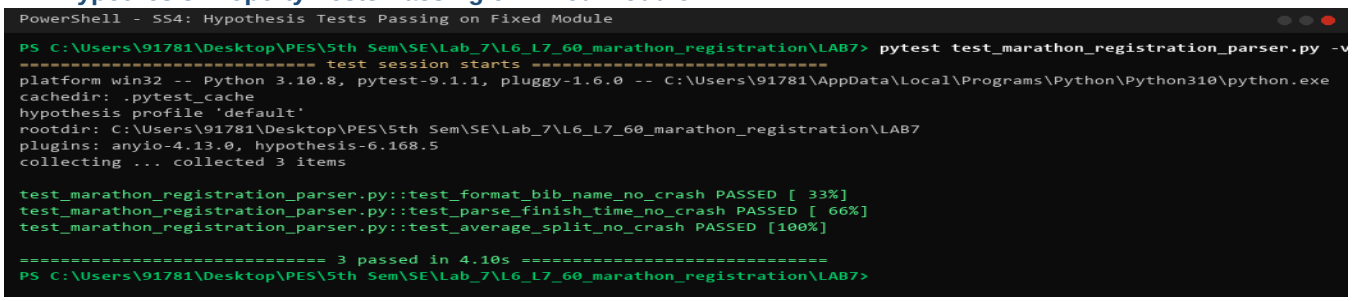


Figure 4: SS4 — All three Hypothesis tests pass completely (100% pass rate) with 0 crashes across extensive fuzzing iterations.

4. Bug Analysis and Defensive Programming Fixes

Function	Identified Vulnerability / Crash Case	Defensive Fix Applied
format_bib_name(name)	Passing None or non-string causes AttributeError: 'NoneType' object has no attribute 'strip'.	Checked isinstance(name, str) and wrapped in try-except; returns empty string "" safely for invalid input.
parse_finish_time(text)	Inputs without exactly two colons (e.g. "", "abc", None) raise ValueError or AttributeError. Non-numeric values raise ValueError in int().	Verified string type, split into exactly 3 parts (hours, minutes, seconds), validated integer bounds, and returned None on failure.
average_split(text)	Empty input ("") yields empty list [], raising ZeroDivisionError. None raises AttributeError. Non-integers raise ValueError.	Verified string type, ensured list is non-empty before division, verified integer splits, and safely returned None on error.

5. Answers to Lab Reflection Questions

Q1. What types of bugs and edge cases did Hypothesis uncover that traditional unit tests often miss?

Answer: Traditional manual unit tests generally focus on predictable "happy paths" and common boundaries specified in requirements. In contrast, Hypothesis generates random and malformed inputs from strategies like st.text() | st.none(). It automatically uncovered:
• **Null pointers / None references:** Passing None caused immediate AttributeError exceptions on .strip() and .split().
• **Empty strings and delimiter-only strings:** Passing "" or ":" caused unpack mismatch ValueError in parse_finish_time and ZeroDivisionError in average_split due to empty lists.
• **Non-numeric characters:** Passing non-digits caused int() parsing to throw uncaught ValueError.

Q2. How did you harden the functions using defensive programming?

Answer: We implemented multiple layers of defensive programming:
1. **Strict Type Verification:** Added isinstance(..., str) guards so non-string objects (including None) are intercepted prior to calling string methods.
2. **Structural & Format Validation:** In parse_finish_time, we verified that splitting on colons yields exactly three non-empty components before unpacking.
3. **Zero-Division Prevention:** In average_split, we validated that the token list is non-empty prior to calculating the average.
4. **Safe Fallback & Exception Trapping:** Enclosed parsing logic within structured try-except blocks to return safe default values (e.g. "" or None) without interrupting system execution.

Q3. Why is property-based fuzz testing critical for software security and quality assurance?

Answer: Software systems regularly ingest untrusted input from external users, network interfaces, and APIs. Unhandled exceptions in production can cause:
• **Denial of Service (DoS):** A single malformed request can crash worker processes or cause entire application outages.
• **Security Vulnerabilities:** Unhandled edge cases can trigger unexpected execution states, bypass authentication checks, or leak internal stack traces.
Property-based fuzzing tests overarching invariants—such as *"no input must ever cause a crash"*. Furthermore, Hypothesis automatically performs **test-case shrinking** to provide minimal counterexamples, dramatically accelerating debugging.

6. Completed Deliverable Files Summary

File	Status	Description
marathon_registration_parser.py	Completed & Verified	Hardened parser module; all 3 functions handle any input without crashing.
test_marathon_registration_parser.py	Completed & Verified	Hypothesis fuzz test suite with try/except assertions covering all 3 functions.
PES1UG25AM814_Shridhar_Golipalle_LAB7.pdf	Generated & Ready	Comprehensive submission document containing SS1-SS4, bug analysis, and reflections.